

FastAPI Course

From Flask REST APIs to a Production-Ready "Test API"

Stack: FastAPI • Pydantic • SQLAlchemy ORM • Alembic • JWT Auth • pytest

DB: SQLite (dev) → Postgres (switch by changing `DATABASE_URL`) `00:00`

Cover Page

What you will build

A full-featured API called **Test API** with:

- Registration + Login (JWT)
 - Users + Tests resources
 - Ownership & admin permissions
 - Pagination
 - File upload
 - Background tasks
 - CORS
 - Automated docs (`/docs` , `/redoc`)
 - Automated tests (pytest)
-

Table of Contents

1. Course Setup & Tooling

2. FastAPI Fundamentals (Flask → FastAPI mental model)
3. Project Structure (Production-ready layout)
4. Configuration with Environment Variables
5. SQLAlchemy ORM Setup (Sessions, Base, DI)
6. Alembic Migrations (Schema evolution)
7. Models (User, Test)
8. Pydantic Schemas (Validation + Serialization)
9. CRUD Layer (Clean DB access)
10. Security (Hashing + JWT)
11. Auth Dependencies (Get current user)
12. API Routes (Auth, Users, Tests)
13. Advanced Features (Background tasks, Uploads, Pagination patterns)
14. Testing (pytest + TestClient)
15. Postgres Switch (No code changes, config only)
16. Final Project Checklist + Exam Questions
17. Production Notes

Design note (PDF export): Use a monospace font for code blocks, and keep headings as-is for auto-generated PDF bookmarks.

1) Course Setup & Tooling

Goal

Set up an environment that supports a modern FastAPI codebase with migrations, auth, and tests. 00:00

Install

Create and activate a virtual environment (venv), then run:

```
bash
```

```

pip install fastapi uvicorn sqlalchemy alembic python-dotenv \
python-jose[cryptography] passlib[bcrypt] python-multipart \
pydantic-settings pytest httpx
``` [[SRC:2]]

Practical Questions
1. What is `uvicorn` responsible for compared to Flask's built-in server?
2. Why do we add `alembic` instead of creating tables manually forever?

2) FastAPI Fundamentals (Flask Mental Model → FastAPI)

Goal
Build a minimal API and understand why FastAPI feels different than Flask. [[SRC:3]]

Key Concepts
- Path operations: `@app.get`, `@app.post`, etc.
- Automatic interactive docs
- Pydantic validation (request + response)
- Dependency Injection (DI) (later used for DB & auth)

Build: Minimal App
Create `app/main.py`:


```

python
from fastapi import FastAPI

app = FastAPI(title="Test API", version="1.0.0")

@app.get("/")
def health():
    return {"status": "ok"}

```


```

Run:

```
bash
```

```
uvicorn app.main:app --reload
```

Open:

- `http://127.0.0.1:8000/`
- `http://127.0.0.1:8000/docs`
- `http://127.0.0.1:8000/redoc`

## Diagram: Request Flow (High Level)

text

```

Client
|
| HTTP request
v
FastAPI route function -----> Pydantic validates input
|
| returns dict / model
v
Pydantic serializes output -----> OpenAPI docs auto-updated
|
v
Client receives JSON response

```

## Practical Questions

1. In Flask, how did you validate request JSON? In FastAPI, what happens automatically?
2. What does the OpenAPI schema enable besides Swagger UI?

## 3) Project Structure (Production-Friendly)

### Goal

Move from “single file app” to a maintainable, scalable layout. 00:00

### Recommended Structure

text

```
app/
 main.py
 core/
 config.py
 security.py
 db/
 base.py
 session.py
 models/
 user.py
 test.py
 schemas/
 user.py
 test.py
 token.py
 crud/
 user.py
 test.py
 api/
 deps.py
 routes/
 auth.py
 users.py
 tests.py
 tests/
 test_auth.py
 test_tests.py
alembic/
alembic.ini
.env
```

## Diagram: Layered Architecture

text

```
Routes (API layer)
|
v
Dependencies (DI): DB session, current user
|
v
CRUD layer (DB operations)
|
v
SQLAlchemy models + DB
```

## Practical Questions

1. Why split "models" vs "schemas"?
2. In Flask, where did you put decorators/middleware? What is the FastAPI equivalent?

## 4) Configuration with Environment Variables

### Goal

Make the app configurable per environment (dev, staging, prod) and DB-switchable. 00:00

**Create** `app/core/config.py`

python

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
 app_name: str = "Test API"
 environment: str = "dev"
 secret_key: str = "change_me"
 access_token_expire_minutes: int = 60

 # SQLite for dev; swap to Postgres later
 database_url: str = "sqlite:///./test.db"

 class Config:
 env_file = ".env"

settings = Settings()
```

## Example `.env`

env

```
ENVIRONMENT=dev
SECRET_KEY=super-secret-change-this
DATABASE_URL=sqlite:///./test.db
```

## Practical Questions

1. Why should `SECRET_KEY` never be committed?
2. What's the advantage of a `DATABASE_URL` string?

## 5) SQLAlchemy ORM Setup (No raw SQL)

### Goal

Create DB engine + session factory and expose a DB session dependency via DI. 00:00



## Create `app/db/session.py`

python

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.core.config import settings

connect_args = {}
if settings.database_url.startswith("sqlite"):
 connect_args = {"check_same_thread": False}

engine = create_engine(settings.database_url, connect_args=connect_args)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

## Create `app/db/base.py`

python

```
from sqlalchemy.orm import declarative_base

Base = declarative_base()
```

## Create `app/api/deps.py` (DB session dependency)

python

```
from app.db.session import SessionLocal

def get_db():
 db = SessionLocal()
 try:
 yield db
 finally:
 db.close()
```

## Practical Questions

1. Why do we `yield` the session rather than return it?
2. What does `autoflush=False` change in practice?

---

## 6) Alembic Migrations (Schema Evolution)

### Goal

Manage schema changes over time using migrations (team-friendly + production-safe).

00:00

### Initialize

```
bash
```

```
alembic init alembic
```

### Configure URL

Update Alembic to use `settings.database_url` in `alembic/env.py`.

### Workflow (Repeatable)

```
bash
```

```
alembic revision --autogenerate -m "create users and tests"
alembic upgrade head
```

## Practical Questions

1. Why are migrations better than `Base.metadata.create_all()` for teams?
2. How do migrations help when switching to Postgres?

## 7) Models (User + Test)

### Goal

Define DB tables with relationships using SQLAlchemy ORM. 00:00

### Create `app/models/user.py`

python

```
from sqlalchemy import Column, Integer, String, Boolean, DateTime
from datetime import datetime
from app.db.base import Base

class User(Base):
 __tablename__ = "users"

 id = Column(Integer, primary_key=True, index=True)
 email = Column(String(255), unique=True, index=True, nullable=False)
 full_name = Column(String(255), nullable=False)
 hashed_password = Column(String(255), nullable=False)
 is_active = Column(Boolean, default=True)
 is_admin = Column(Boolean, default=False)
 created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
```

### Create `app/models/test.py`

python

```

from sqlalchemy import Column, Integer, String, Boolean, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from datetime import datetime
from app.db.base import Base

class Test(Base):
 __tablename__ = "tests"

 id = Column(Integer, primary_key=True, index=True)
 title = Column(String(255), nullable=False, index=True)
 description = Column(String(1000), nullable=True)
 is_published = Column(Boolean, default=False, nullable=False)

 owner_id = Column(Integer, ForeignKey("users.id"), nullable=False)
 owner = relationship("User")

 created_at = Column(DateTime, default=datetime.utcnow, nullable=False)

```

## Ensure models are importable for migrations

Create `app/models/__init__.py`:

```
python
```

```

from app.models.user import User
from app.models.test import Test

```

## Practical Questions

1. Why store `hashed_password` instead of `password`?
2. When would you index `title`?

## 8) Pydantic Schemas (Validation + Serialization)

### Goal

Define request/response contracts cleanly (no leaking internal DB fields). 00:00

## Create `app/schemas/user.py`

python

```
from pydantic import BaseModel, EmailStr

class UserBase(BaseModel):
 email: EmailStr
 full_name: str

class UserCreate(UserBase):
 password: str

class UserOut(UserBase):
 id: int
 is_active: bool
 is_admin: bool

class Config:
 from_attributes = True
```

## Create `app/schemas/test.py`

python

```

from pydantic import BaseModel
from typing import Optional

class TestBase(BaseModel):
 title: str
 description: Optional[str] = None

class TestCreate(TestBase):
 pass

class TestUpdate(BaseModel):
 title: Optional[str] = None
 description: Optional[str] = None
 is_published: Optional[bool] = None

class TestOut(TestBase):
 id: int
 is_published: bool
 owner_id: int

class Config:
 from_attributes = True

```

## Practical Questions

1. Why is `TestUpdate` optional fields-only?
2. What does `from_attributes=True` enable?

## 9) CRUD Layer (DB Operations)

### Goal

Keep routes thin and reuse DB logic safely and consistently. 00:00

**Create** `app/crud/user.py`

python

```
from sqlalchemy.orm import Session
from app.models.user import User
from app.schemas.user import UserCreate
from app.core.security import get_password_hash

def get_by_email(db: Session, email: str) -> User | None:
 return db.query(User).filter(User.email == email).first()

def create(db: Session, user_in: UserCreate) -> User:
 user = User(
 email=user_in.email,
 full_name=user_in.full_name,
 hashed_password=get_password_hash(user_in.password),
)
 db.add(user)
 db.commit()
 db.refresh(user)
 return user
```

**Create** `app/crud/test.py`

python

```

from sqlalchemy.orm import Session
from app.models.test import Test
from app.schemas.test import TestCreate, TestUpdate

def create(db: Session, owner_id: int, test_in: TestCreate) -> Test:
 test = Test(owner_id=owner_id, **test_in.model_dump())
 db.add(test)
 db.commit()
 db.refresh(test)
 return test

def get(db: Session, test_id: int) -> Test | None:
 return db.query(Test).filter(Test.id == test_id).first()

def list(db: Session, skip: int = 0, limit: int = 20) -> list[Test]:
 return db.query(Test).offset(skip).limit(limit).all()

def update(db: Session, test: Test, test_in: TestUpdate) -> Test:
 data = test_in.model_dump(exclude_unset=True)
 for k, v in data.items():
 setattr(test, k, v)
 db.add(test)
 db.commit()
 db.refresh(test)
 return test

def delete(db: Session, test: Test) -> None:
 db.delete(test)
 db.commit()

```

## Practical Questions

1. Why use `exclude_unset=True` for PATCH-like updates?
2. What does `db.refresh()` do?



## 10) Security (Password Hashing + JWT)

### Goal

Implement secure authentication: hash passwords, issue JWT access tokens. 00:00

### Create `app/core/security.py`

python

```
from datetime import datetime, timedelta, timezone
from jose import jwt
from passlib.context import CryptContext
from app.core.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
ALGORITHM = "HS256"

def verify_password(plain_password: str, hashed_password: str) -> bool:
 return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
 return pwd_context.hash(password)

def create_access_token(subject: str, expires_minutes: int | None = None) -> str:
 if expires_minutes is None:
 expires_minutes = settings.access_token_expire_minutes
 expire = datetime.now(timezone.utc) + timedelta(minutes=expires_minutes)
 to_encode = {"sub": subject, "exp": expire}
 return jwt.encode(to_encode, settings.secret_key, algorithm=ALGORITHM)
```

### Create `app/schemas/token.py`

python

```
from pydantic import BaseModel

class Token(BaseModel):
 access_token: str
 token_type: str = "bearer"
```

## Diagram: Auth Flow

text

```
Register (email + password)
 |
 v
Password hashed in DB
 |
 v
Login (OAuth2 password form)
 |
 v
JWT access_token returned
 |
 v
Client sends: Authorization: Bearer <token>
 |
 v
Protected endpoints verify JWT -> load current user
```

## Practical Questions

1. Why is JWT "stateless"? What do you lose vs server sessions?
2. What claim stores identity here? ( `sub` )

## 11) Auth Dependencies (Get Current User)

### Goal

Protect endpoints using dependencies and token validation. 00:00

**Extend** `app/api/deps.py`

python

```

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError
from sqlalchemy.orm import Session

from app.core.security import ALGORITHM
from app.core.config import settings
from app.models.user import User
from app.api.deps import get_db # if circular, split into separate module

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

def get_current_user(
 db: Session = Depends(get_db),
 token: str = Depends(oauth2_scheme),
) -> User:
 credentials_exception = HTTPException(
 status_code=status.HTTP_401_UNAUTHORIZED,
 detail="Could not validate credentials",
 headers={"WWW-Authenticate": "Bearer"},
)
 try:
 payload = jwt.decode(token, settings.secret_key, algorithms=[ALGORITHM])
 email: str | None = payload.get("sub")
 if email is None:
 raise credentials_exception
 except JWTError:
 raise credentials_exception

 user = db.query(User).filter(User.email == email).first()
 if not user:
 raise credentials_exception
 return user

```

## Practical Questions

1. Why do we return `WWW-Authenticate: Bearer` ?
2. What's the difference between authentication and authorization?

## 12) API Routes (Auth, Users, Tests)

### Goal

Create routers like Flask Blueprints, but with typed inputs/outputs and DI. 00:00

### Create `app/api/routes/auth.py`

python

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session

from app.api.deps import get_db
from app.crud.user import get_by_email
from app.core.security import verify_password, create_access_token
from app.schemas.token import Token

router = APIRouter(prefix="/auth", tags=["auth"])

@router.post("/login", response_model=Token)
def login(
 db: Session = Depends(get_db),
 form_data: OAuth2PasswordRequestForm = Depends(),
):
 user = get_by_email(db, form_data.username)
 if not user or not verify_password(form_data.password, user.hashed_password):
 raise HTTPException(
 status_code=status.HTTP_400_BAD_REQUEST,
 detail="Incorrect email or password",
)
 token = create_access_token(subject=user.email)
 return {"access_token": token, "token_type": "bearer"}
```

### Create `app/api/routes/users.py`

python

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from app.api.deps import get_db
from app.schemas.user import UserCreate, UserOut
from app.crud.user import get_by_email, create

router = APIRouter(prefix="/users", tags=["users"])

@router.post("", response_model=UserOut, status_code=201)
def register_user(user_in: UserCreate, db: Session = Depends(get_db)):
 existing = get_by_email(db, user_in.email)
 if existing:
 raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Email already
registered")
 return create(db, user_in)
```

**Create** `app/api/routes/tests.py`

python

```

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from app.api.deps import get_db, get_current_user
from app.models.user import User
from app.schemas.test import TestCreate, TestUpdate, TestOut
from app.crud import test as crud_test

router = APIRouter(prefix="/tests", tags=["tests"])

@router.post("", response_model=TestOut, status_code=201)
def create_test(
 test_in: TestCreate,
 db: Session = Depends(get_db),
 current_user: User = Depends(get_current_user),
):
 return crud_test.create(db, owner_id=current_user.id, test_in=test_in)

@router.get("", response_model=list[TestOut])
def list_tests(skip: int = 0, limit: int = 20, db: Session = Depends(get_db)):
 return crud_test.list(db, skip=skip, limit=limit)

@router.get("/{test_id}", response_model=TestOut)
def get_test(test_id: int, db: Session = Depends(get_db)):
 test = crud_test.get(db, test_id)
 if not test:
 raise HTTPException(status_code=404, detail="Test not found")
 return test

@router.patch("/{test_id}", response_model=TestOut)
def update_test(
 test_id: int,
 test_in: TestUpdate,
 db: Session = Depends(get_db),
 current_user: User = Depends(get_current_user),
):
 test = crud_test.get(db, test_id)
 if not test:
 raise HTTPException(status_code=404, detail="Test not found")
 if test.owner_id != current_user.id and not current_user.is_admin:
 raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not enough
permissions")
 return crud_test.update(db, test, test_in)

@router.delete("/{test_id}", status_code=204)
def delete_test(

```

```
test_id: int,
db: Session = Depends(get_db),
current_user: User = Depends(get_current_user),
):
 test = crud_test.get(db, test_id)
 if not test:
 raise HTTPException(status_code=404, detail="Test not found")
 if test.owner_id != current_user.id and not current_user.is_admin:
 raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not enough
permissions")
 crud_test.delete(db, test)
 return None
```

## Practical Questions

1. Why do we use routers with prefixes and tags?
2. Why should delete return 204 with no JSON?

## 13) Main App Wiring (Routers, CORS, DB Base)

### Goal

Connect everything so the API runs end-to-end. 00:00

Create `app/main.py`

python

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.core.config import settings
from app.db.base import Base
from app.db.session import engine
from app.api.routes import auth, users, tests

def create_app() -> FastAPI:
 app = FastAPI(title=settings.app_name)

 app.add_middleware(
 CORSMiddleware,
 allow_origins=["*"],
 allow_credentials=True,
 allow_methods=["*"],
 allow_headers=["*"],
)

 # Dev-only convenience (prefer Alembic in real setups)
 Base.metadata.create_all(bind=engine)

 app.include_router(auth.router)
 app.include_router(users.router)
 app.include_router(tests.router)

 @app.get("/")
 def health():
 return {"status": "ok", "app": settings.app_name}

 return app

app = create_app()

```

Run:

```
bash
```

```
uvicorn app.main:app --reload
```



## Practical Questions

1. When should you remove `create_all()` and rely only on Alembic?
2. What is CORS and why might it break your frontend?

## 14) Advanced Features ("Almost All Features" You'll Actually Use)

### Goal

Add practical features frequently required in real APIs. 00:00

### 14.1 Background Tasks

python

```
from fastapi import BackgroundTasks

@router.post("/notify")
def notify(background_tasks: BackgroundTasks):
 def send_email_sim():
 print("Sending email in background...")

 background_tasks.add_task(send_email_sim)
 return {"queued": True}
```

### 14.2 File Upload

python

```
from fastapi import UploadFile, File

@router.post("/upload")
def upload(file: UploadFile = File(...)):
 return {"filename": file.filename, "content_type": file.content_type}
```

## 14.3 Pagination Pattern

Already supported via:

- `GET /tests?skip=0&limit=20`

### Extension ideas (exercise)

- `q` search: `?q=math`
- sort: `?sort=created_at&direction=desc`

## Practical Questions

1. When are background tasks appropriate vs a job queue (Celery/RQ)?
2. Why use `UploadFile` for large files?
3. Why is cursor pagination better than offset pagination for huge datasets?

---

## 15) Testing (pytest + FastAPI TestClient)

---

### Goal

Write tests for auth and a CRUD flow. 00:00

Create `app/tests/conftest.py`

python

```
import pytest
from fastapi.testclient import TestClient
from app.main import app

@pytest.fixture()
def client():
 return TestClient(app)
```

## Create `app/tests/test_auth.py`

python

```
def test_register_and_login(client):
 r = client.post("/users", json=
{"email":"a@a.com","full_name":"A","password":"pass123"})
 assert r.status_code == 201

 r = client.post("/auth/login", data={"username":"a@a.com","password":"pass123"})
 assert r.status_code == 200
 assert "access_token" in r.json()
```

## Create `app/tests/test_tests.py`

python

```
def test_create_test_requires_auth(client):
 r = client.post("/tests", json={"title":"T1","description":"D"})
 assert r.status_code in (401, 403)

def test_create_list_test_flow(client):
 client.post("/users", json={"email":"b@b.com","full_name":"B","password":"pass123"})
 r = client.post("/auth/login", data={"username":"b@b.com","password":"pass123"})
 token = r.json()["access_token"]

 headers = {"Authorization": f"Bearer {token}"}
 r = client.post("/tests", json={"title":"My Test","description":"Desc"},
headers=headers)
 assert r.status_code == 201

 r = client.get("/tests")
 assert r.status_code == 200
 assert isinstance(r.json(), list)
```

Run:

bash

```
pytest -q
```

## Practical Questions

1. Why test unauthorized access explicitly?
2. How would you isolate tests from your dev DB?

## 16) Postgres Switch (SQLAlchemy URL Only)

### Goal

Move from SQLite to Postgres without changing ORM code. 00:00

### Change `.env`

```
env
```

```
DATABASE_URL=postgresql+psycopg2://user:password@localhost:5432/test_api
```

Install driver:

```
bash
```

```
pip install psycopg2-binary
```

## Practical Questions

1. Why avoid SQLite-specific assumptions in production?
2. What migrations do you run when switching DBs?

## 17) Final Project Checklist — Fully Functional “Test API”

---

### Endpoints

- `GET /health`
- `POST /users` register
- `POST /auth/login` (OAuth2 password flow)
- `POST /tests` create (auth required)
- `GET /tests` list (public)
- `GET /tests/{id}` read
- `PATCH /tests/{id}` update (owner/admin)
- `DELETE /tests/{id}` delete (owner/admin)
- Optional:
  - `POST /tests/upload` file upload
- `POST /tests/notify` background task demo

### Feature Checklist

- ☐ Pydantic request validation
- ☐ Response models (never leak `hashed_password`)
- ☐ SQLAlchemy ORM only (no raw SQL)
- ☐ Routers + DI (DB + auth)
- ☐ JWT auth + bcrypt hashing
- ☐ Authorization checks (owner/admin)
- ☐ Pagination ( `skip`, `limit` )
- ☐ Tests (auth + CRUD)
- ☐ CORS configured

- [ ] Env-based config
  - [ ] Postgres-ready DB URL
- 

## 18) Exam Questions (Answer after building)

---

1. Why separate schemas from models?
  2. How does dependency injection work in FastAPI (DB + auth examples)?
  3. What security mistakes happen if you return a SQLAlchemy User model directly?
  4. How would you implement refresh tokens (high-level)?
  5. How would you add rate limiting (high-level)?
  6. What changes when deploying behind a reverse proxy?
- 

## 19) Production Notes (Minimal)

---

- Use environment variables for secrets.
- Prefer Alembic migrations over `create_all()` in long-lived apps.
- Put the API behind a reverse proxy in real deployments.
- Add logging + monitoring for real operations.
- Consider async DB later (after mastering sync ORM patterns).